

电子科技大学

实 验 报 告

(实验) 课程名称 M2M 通信实验报告

基于 NodeMCU-32 的猫粮重量监测系统

Cat Food Weight Monitoring System Based on M2M Communication

小组成员 (Group11)	赵杭键-202544060167 贾子涵-202544060121 周航宇-202544060168
----------------	--

说明：本报告整理了从开发环境搭建、终端烧录、MQTT 联调到 PC Web 平台实现的完整实验流程。

终端开发工具采用 VS Code + PlatformIO。

摘要

本实验围绕宠物喂养场景设计并实现一个猫粮重量监测 M2M 系统。系统以 NodeMCU-32 为终端控制器，FSR 薄膜压力传感器采集猫粮碗底座压力变化，终端通过 GPIO34/ADC 读取传感器分压信号，完成滤波、去皮、重量换算与阈值判断。终端通过 Wi-Fi 接入局域网，并使用 MQTT 协议将重量数据和设备状态发布至 PC 端 Mosquitto Broker。PC 端使用 Python Flask 服务订阅 MQTT 数据，写入 SQLite 数据库，并通过 Web 前端展示当前重量、ADC 原始值、报警阈值、设备在线状态、实时趋势和告警记录，同时支持远程下发采集间隔、报警阈值和清零指令。

实验过程完成了 VS Code + PlatformIO 项目创建、NodeMCU-32 固件编译烧录、Mosquitto 监听配置、MQTT 数据订阅验证、Flask 服务端开发、SQLite 数据存储和前端页面联调。结果表明，系统能够实现 M2M 中“终端感知—网络传输—平台处理—应用展示—远程控制”的闭环。

关键词：M2M；NodeMCU-32；PlatformIO；MQTT；Mosquitto；FSR；Flask；SQLite；Web 前端

目录

目录

基于 NodeMCU-32 的猫粮重量监测系统 1

摘要 2

目录 3

1 实验目的与总体思路 5

2 实验设备与开发环境 5

 2.1 硬件与网络环境 5

 2.2 软件环境 5

3 系统架构与硬件连接 6

 3.1 M2M 功能映射 6

 3.2 FSR 与 NodeMCU-32 接线 7

 3.3 UML 设计 7

4 VS Code + PlatformIO 终端工程搭建 8

 4.1 PlatformIO 安装与项目创建 8

 4.2 platformio.ini 配置 9

 4.3 工程目录 9

5 NodeMCU-32 程序设计与烧录 9

 5.1 终端程序功能 9

 5.2 关键终端代码逻辑 10

 5.3 编译、烧录与串口监视 10

6 Mosquitto Broker 安装与配置 11

 6.1 问题现象 11

 6.2 正确配置 11

7 MQTT 联调与数据验证 12

 7.1 Broker 日志验证 12

 7.2 订阅重量数据 12

8 Python Flask + SQLite 服务端平台 13

 8.1 PC 平台目录 13

 8.2 服务端功能 13

 8.3 运行服务端 13

9 Web 前端页面设计与运行效果 14

 9.1 页面模块 14

 9.2 前端刷新与控制逻辑 14

10 远程控制与功能测试 15

 10.1 控制指令格式 15

10.2 功能测试结果 15

11 问题排查与解决过程 15

11.1 PlatformIO 首次安装较慢 15

11.2 mqtt_server 变量未定义 15

11.3 NodeMCU-32 MQTT rc=-2 15

11.4 1883 端口被占用 15

12 实验结果与总结 16

附录 A 关键配置与代码片段 16

A.1 platformio.ini 16

A.2 Mosquitto 配置 16

A.3 Flask 服务端核心处理 16

A.4 前端主要 API 17

1 实验目的与总体思路

本实验目标是完成一个可运行、可演示、可扩展的猫粮重量监测系统。系统以 NodeMCU-32 作为终端节点，采集 FSR 传感器的压力变化并换算为猫粮剩余重量，再通过 MQTT 发布/订阅机制与 PC 端服务平台通信。PC 平台负责数据接收、数据库存储、低粮告警、历史趋势展示以及远程配置。

- 掌握 VS Code + PlatformIO 下 NodeMCU-32 工程的创建、编译、烧录和串口监视流程。
- 掌握 FSR 分压采样、ADC 滤波、去皮和重量换算的终端程序设计方法。
- 掌握 Mosquitto MQTT Broker 的局域网监听配置与发布/订阅验证方法。
- 掌握 Python Flask 服务订阅 MQTT、写入 SQLite 并提供 Web API 的实现方式。
- 搭建简洁清爽的 Web 前端页面，实现实时状态展示、趋势图、告警记录和控制指令下发。

本报告按真实实验操作顺序整理，从环境安装开始，记录关键命令、配置、截图和排查过程，便于复现实验。

场景：宠物主人需远程了解猫粮碗内剩余重量，当余量过低时收到提醒，并能动态调整报警阈值。

用例：

- 用户通过 PC 查看当前猫粮重量、历史数据曲线。
- 系统自动定期采集重量，存储记录，检测余量是否低于阈值并告警。
- 可远程配置采集间隔、重量报警阈值。
- 设备上线时自动注册，服务端追踪其在线状态。

2 实验设备与开发环境

2.1 硬件与网络环境

类别	内容	说明
终端控制器	NodeMCU-32 / ESP32	负责 ADC 采样、Wi-Fi 连接、MQTT 通信与控制指令处理
传感器	FSR 薄膜压力传感器	放置在猫粮碗底座，用于感知压力变化
分压元件	10kΩ 固定电阻	与 FSR 构成分压电路，输出送入 GPIO34 ADC
服务端	Windows PC	运行 Mosquitto、Flask、SQLite 与 Web 页面
网络	iPhone 热点 hajimi / Wi-Fi 局域网	PC 与 NodeMCU-32 位于同一网段，实验中终端 IP 为 172.20.10.11

2.2 软件环境

软件	用途	说明
Visual Studio Code	代码编辑与工程入口	本实验不使用 Arduino IDE
PlatformIO IDE 插件	ESP32 工程管理、编译、上传和串口监视	板卡选择 NodeMCU-32S
PlatformIO Core	底层编译与工具链管理	自动安装 espressif32、toolchain、esptool 等依赖
Mosquitto 2.1.2	MQTT Broker	配置 listener 1883 0.0.0.0 允许局域网设备连接
Python + Flask	PC 数据服务与 Web API	订阅 MQTT 数据并提供 /api/latest、/api/history 等接口
SQLite	本地数据库	保存重量记录、设备状态与告警记录

需要特别说明的是，PlatformIO 配置中使用的 framework = arduino 是 ESP32 的 Arduino 开发框架，用于调用 WiFi、PubSubClient 和 ArduinoJson 等库；它并不表示使用 Arduino IDE。本实验的开发、编译和下载均在 VS Code + PlatformIO 中完成。

3 系统架构与硬件连接

- 终端层：ESP32 NodeMCU-32 + 薄膜压力传感器 (FSR)，直接接 ADC 引脚。
- 通信层：Wi-Fi 连接手机热点/路由器；MQTT 协议（Mosquitto Broker，QoS=1）。
- 服务层：PC 运行 Mosquitto + Python 程序（数据存储/控制接口），SQLite 记录设备状态。
- 主题规划：
 - pet/weight/data – ESP32 发布重量(kg)、时间戳。
 - pet/device/control – 服务端下发配置指令（JSON 格式）。
 - pet/device/status – ESP32 发送心跳/配置确认。
- ESP32 上的角色：MQTT 客户端 (Client)
- PC 上的角色：MQTT Broker (服务器)

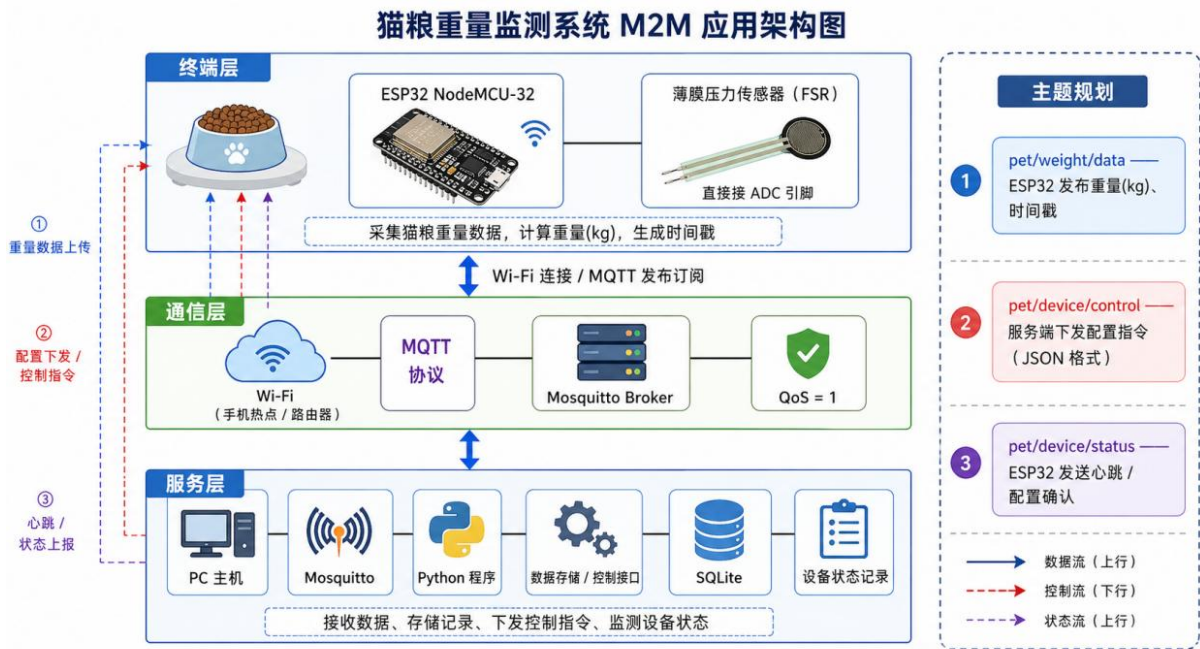
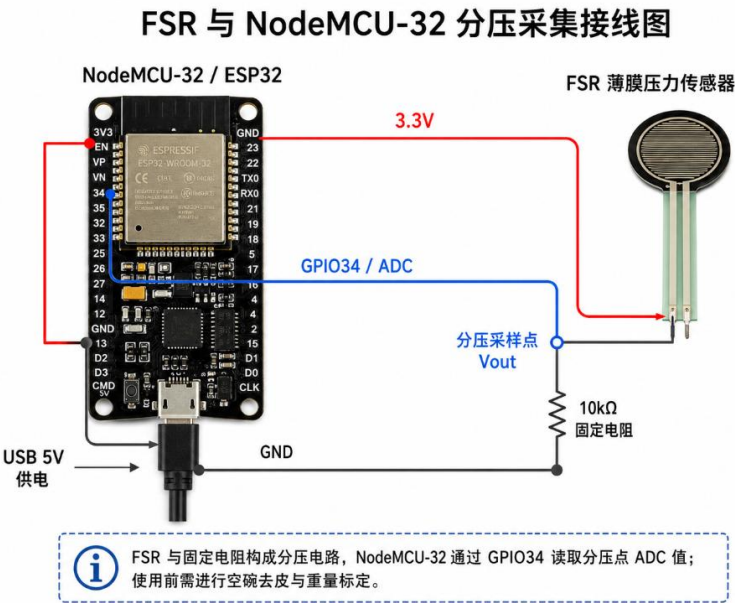


图 1 猫粮重量监测系统 M2M 应用架构图

3.1 M2M 功能映射

M2M 层次	本系统对应部分	功能说明
感知对象	猫粮碗、FSR 压力传感器	将猫粮余量变化转化为压力和模拟电压变化
M2M 终端	NodeMCU-32	ADC 采集、滤波换算、MQTT 发布和控制指令处理
通信网络	Wi-Fi + TCP/IP + MQTT	完成终端与 PC 平台之间的双向消息传输
中间件/平台	Mosquitto + Flask + SQLite	转发消息、存储数据、判断告警和提供 API
应用层	Web 前端页面	显示重量、趋势、状态和告警，并下发配置

3.2 FSR 与 NodeMCU-32 接线

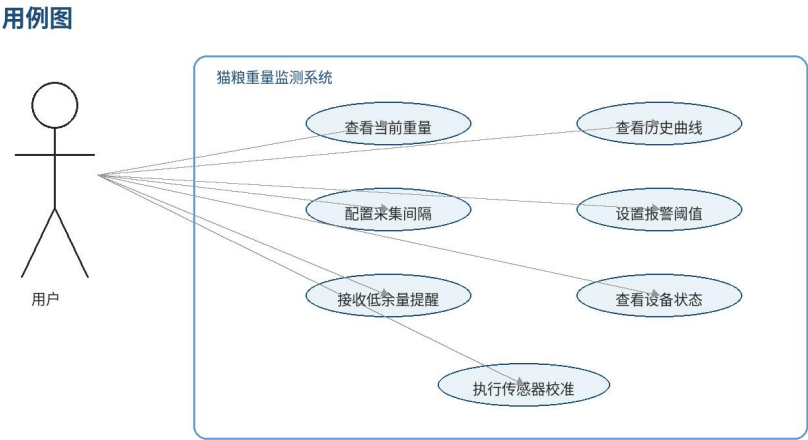


连接点	接线方式	说明
FSR 一端	3.3V	传感器分压电路上拉端
FSR 另一端	GPIO34 与 10kΩ 电阻连接点	形成 ADC 采样节点
10kΩ 电阻另一端	GND	分压电路下拉端
ADC 输入	GPIO34 / ADC1_CH6	读取分压电压，换算为 raw_adc
供电	USB 5V	为 NodeMCU-32 供电和串口调试

称重前需要空碗去皮，记录零点 ADC 值；随后放入不同重量猫粮进行多点标定，得到 ADC 与重量之间的换算参数。为了降低抖动，每次上报前采用多次采样平均。

3.3 UML 设计

为说明系统功能边界、对象结构和运行过程，本报告绘制用例图、类图和时序图。



类图

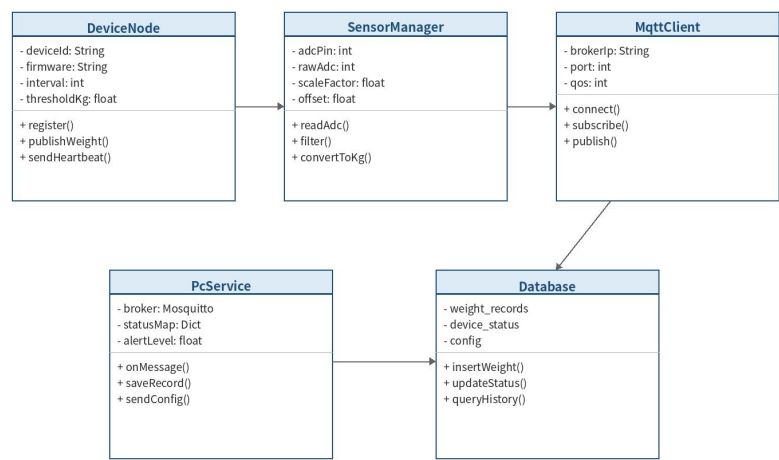


图 4 系统类图

时序图



图 5 系统时序图

4 VS Code + PlatformIO 终端工程搭建

4.1 PlatformIO 安装与项目创建

在 VS Code 插件市场安装 PlatformIO IDE，安装完成后通过 PIO Home 新建工程。Project Wizard 中项目结构包括 lib、src 和 platformio.ini，其中 src/main.cpp 存放终端源代码，platformio.ini 存放板卡、框架、串口和依赖库配置。

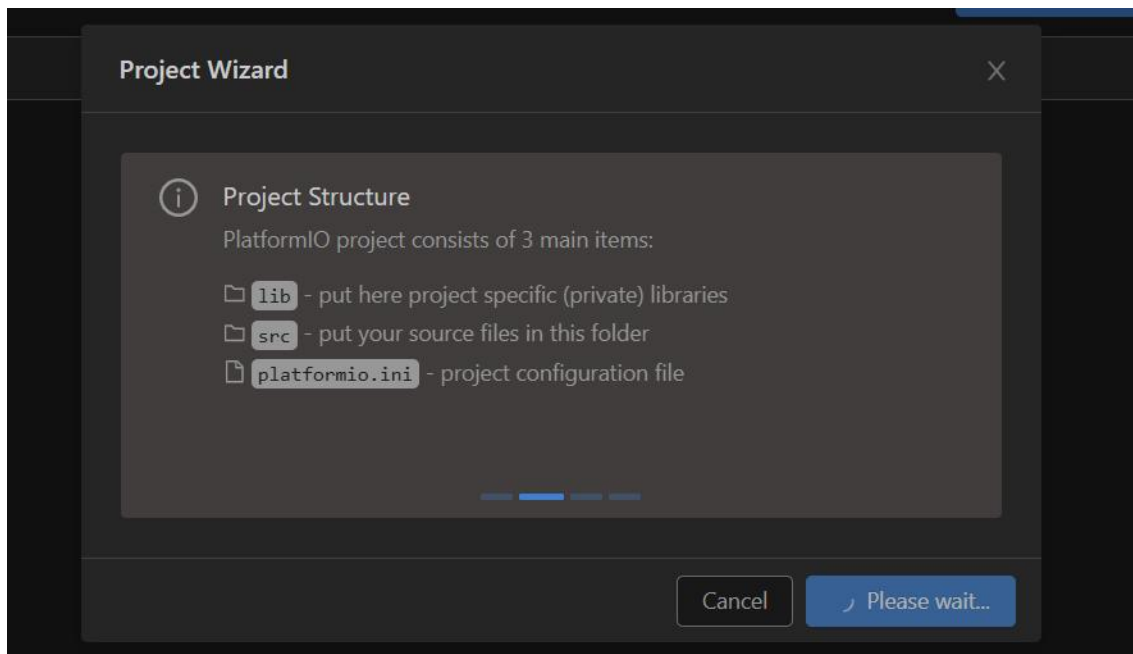


图6 PlatformIO Project Wizard 与项目结构说明

4.2 platformio.ini 配置

本实验采用 NodeMCU-32S 板卡，ESP32 平台，串口监视波特率为 115200，并引入 MQTT 与 JSON 库。配置如下：

```
[env:nodemcu-32s]
platform = espressif32
board = nodemcu-32s
framework = arduino
monitor_speed = 115200

lib_deps =
  knolleary/PubSubClient@^2.8
  bblanchon/ArduinoJson@^6.21.5
```

4.3 工程目录

```
cat_food_monitor
├── platformio.ini
├── src
│   └── main.cpp
├── lib
└── .pio
```

5 NodeMCU-32 程序设计与烧录

5.1 终端程序功能

- 连接 Wi-Fi 热点，获取局域网 IP。
- 连接 PC 端 Mosquitto Broker，客户端 ID 为 cat_bowl_01。
- 订阅 pet/device/control，用于接收采集间隔、阈值和清零指令。
- 通过 GPIO34 采集 FSR 分压后的 ADC 原始值，执行滤波和重量换算。
- 向 pet/weight/data 发布重量数据，向 pet/device/status 发布上线和配置确认状态。
- 若 Wi-Fi 或 MQTT 断开，终端自动重连。

5.2 关键终端代码逻辑

终端基础配置

```
#include <WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>

#define ADC_PIN 34
const char* ssid = "hajimi";
const char* password = "123456789";
const char* mqtt_server = "172.20.10.4";

WiFiClient espClient;
PubSubClient client(espClient);
String deviceId = "cat_bowl_01";
unsigned long sampleInterval = 5000;
float alarmThresholdKg = 0.15;
float zeroAdc = 1200;
float calibrationK = 0.0005;
```

FSR ADC 采样、滤波与重量换算

```
float readAverageAdc() {
    long sum = 0;
    const int sampleCount = 20;
    for (int i = 0; i < sampleCount; i++) {
        sum += analogRead(ADC_PIN);
        delay(5);
    }
    return sum / (float)sampleCount;
}

float readWeightKg(float raw) {
    float weight = (raw - zeroAdc) * calibrationK;
    if (weight < 0) weight = 0;
    return weight;
}
```

重量数据上报

```
void publishWeight() {
    float raw = readAverageAdc();
    float weight = readWeightKg(raw);

    StaticJsonDocument<256> doc;
    doc["device_id"] = deviceId;
    doc["weight_kg"] = weight;
    doc["raw_adc"] = raw;
    doc["alarm_threshold_kg"] = alarmThresholdKg;

    char buffer[256];
    serializeJson(doc, buffer);
    client.publish("pet/weight/data", buffer);
    Serial.println(buffer);
}
```

5.3 编译、烧录与串口监视

终端固件在 PlatformIO 中完成编译和烧录。实验中使用 COM8 端口，烧录过程能够识别 ESP32-D0WD-V3 芯片，固件写入后自动复位。

常用 PlatformIO 命令

```
pio run
pio run --target upload
pio device monitor
```

编译和上传成功后，串口监视器显示 Wi-Fi 连接成功，终端 IP 为 172.20.10.11，并开始连接 MQTT Broker。

6 Mosquitto Broker 安装与配置

6.1 问题现象

首次在 PowerShell 中直接执行 `mosquitto -v` 时出现命令无法识别, 说明 Mosquitto 未加入 PATH 或未通过完整路径调用。随后通过安装目录中的 `mosquitto.exe` 启动 Broker。

```
& "C:\Program Files\mosquitto\mosquitto.exe" -v
```

默认启动时 Mosquitto 进入 local only mode, 仅监听 127.0.0.1:1883, 导致 NodeMCU-32 连接失败, 串口显示 MQTT failed, rc=-2。通过 `netstat` 发现 1883 端口由后台 `mosquitto.exe` 服务占用。

```
netstat -ano | findstr :1883
tasklist /FI "PID eq 15040"
Stop-Service mosquitto
Set-Service mosquitto -StartupType Manual
```

6.2 正确配置

为允许局域网内 NodeMCU-32 连接 PC Broker, 在桌面创建 `mosquitto.conf`, 并配置监听所有网卡地址。

`mosquitto.conf`

```
listener 1883 0.0.0.0
allow_anonymous true
```

以配置文件启动 Mosquitto

```
& "C:\Program Files\mosquitto\mosquitto.exe" -c "$env:USERPROFILE\Desktop\mosquitto.conf" -v
```

启动成功后, Mosquitto 日志显示 Opening ipv4 listen socket on port 1883, 随后 NodeMCU-32 以 `cat_bowl_01` 客户端身份连接 Broker。

```
Windows PowerShell
版权所有 (C) Microsoft Corporation. 保留所有权利。

尝试新的跨平台 PowerShell https://aka.ms/pscore6

PS C:\Windows\system32> Stop-Service mosquitto
PS C:\Windows\system32> Set-Service mosquitto -StartupType Manual
PS C:\Windows\system32> netstat -ano | findstr :1883
PS C:\Windows\system32> notepad "$env:USERPROFILE\Desktop\mosquitto.conf"
PS C:\Windows\system32> & "C:\Program Files\mosquitto\mosquitto.exe" -c "$env:USERPROFILE\Desktop\mosquitto.conf" -v
1779612525: mosquitto version 2.1.2 starting
1779612525: Config loaded from C:\Users\hangj\Desktop\mosquitto.conf.
1779612525: Bridge support available.
1779612525: Persistence support available.
1779612525: TLS support available.
1779612525: TLS-PSK support available.
1779612525: Websockets support available.
1779612525: Opening ipv4 listen socket on port 1883.
1779612525: mosquitto version 2.1.2 running
1779612528: New connection from 172.20.10.11:64721 on port 1883.
1779612528: New client connected from 172.20.10.11:64721 as cat_bowl_01 (p4, cl, k15).
1779612528: No will message specified.
1779612528: Sending CONNACK to cat_bowl_01 (0, 0)
1779612528: Received SUBSCRIBE from cat_bowl_01
1779612528:   pet/device/control (QoS 0)
1779612528: cat_bowl_01 0 pet/device/control
1779612528: Sending SUBACK to cat_bowl_01
1779612528: Received PUBLISH from cat_bowl_01 (d0, q0, r0, m0, 'pet/device/status', ... (100 bytes))
1779612528: Received PUBLISH from cat_bowl_01 (d0, q0, r0, m0, 'pet/weight/data', ... (99 bytes))
1779612533: Received PUBLISH from cat_bowl_01 (d0, q0, r0, m0, 'pet/weight/data', ... (99 bytes))
1779612538: Received PUBLISH from cat_bowl_01 (d0, q0, r0, m0, 'pet/weight/data', ... (99 bytes))
1779612543: Received PUBLISH from cat_bowl_01 (d0, q0, r0, m0, 'pet/weight/data', ... (99 bytes))
1779612543: Received PINGREQ from cat_bowl_01
1779612543: Sending PINGRESP to cat_bowl_01
1779612549: Received PUBLISH from cat_bowl_01 (d0, q0, r0, m0, 'pet/weight/data', ... (98 bytes))
1779612553: Received PUBLISH from cat_bowl_01 (d0, q0, r0, m0, 'pet/weight/data', ... (99 bytes))
1779612558: Received PUBLISH from cat_bowl_01 (d0, q0, r0, m0, 'pet/weight/data', ... (99 bytes))
1779612558: Received PINGREQ from cat_bowl_01
1779612558: Sending PINGRESP to cat_bowl_01
1779612563: Received PUBLISH from cat_bowl_01 (d0, q0, r0, m0, 'pet/weight/data', ... (99 bytes))
1779612568: Received PUBLISH from cat_bowl_01 (d0, q0, r0, m0, 'pet/weight/data', ... (99 bytes))
1779612573: Received PUBLISH from cat_bowl_01 (d0, q0, r0, m0, 'pet/weight/data', ... (99 bytes))
1779612573: Received PINGREQ from cat_bowl_01
1779612573: Sending PINGRESP to cat_bowl_01
1779612578: Received PUBLISH from cat_bowl_01 (d0, q0, r0, m0, 'pet/weight/data', ... (99 bytes))
1779612583: Received PUBLISH from cat_bowl_01 (d0, q0, r0, m0, 'pet/weight/data', ... (98 bytes))
1779612588: Received PUBLISH from cat_bowl_01 (d0, q0, r0, m0, 'pet/weight/data', ... (99 bytes))
1779612588: Received PINGREQ from cat_bowl_01
1779612588: Sending PINGRESP to cat_bowl_01
1779612593: Received PUBLISH from cat_bowl_01 (d0, q0, r0, m0, 'pet/weight/data', ... (99 bytes))
1779612598: Received PUBLISH from cat_bowl_01 (d0, q0, r0, m0, 'pet/weight/data', ... (99 bytes))
1779612603: Received PUBLISH from cat_bowl_01 (d0, q0, r0, m0, 'pet/weight/data', ... (99 bytes))
1779612603: Received PINGREQ from cat_bowl_01
1779612603: Sending PINGRESP to cat_bowl_01
1779612608: Received PUBLISH from cat_bowl_01 (d0, q0, r0, m0, 'pet/weight/data', ... (98 bytes))
```

图 7 Mosquitto 接收 NodeMCU-32 连接、订阅和发布日志

7 MQTT 联调与数据验证

7.1 Broker 日志验证

Mosquitto 控制台日志显示 NodeMCU-32 从 172.20.10.11 接入，并订阅 pet/device/control，随后持续发布 pet/device/status 和 pet/weight/data。关键日志如下：

```
管理员: Windows PowerShell
Windows PowerShell
版权所有 (C) Microsoft Corporation。保留所有权利。

尝试新的跨平台 PowerShell https://aka.ms/pscore6

PS C:\Windows\system32> Stop-Service mosquitto
PS C:\Windows\system32> Set-Service mosquitto -StartupType Manual
PS C:\Windows\system32> netstat -ano | findstr :1883
PS C:\Windows\system32> notepad %env:USERPROFILE\Desktop\mosquitto.conf%
PS C:\Windows\system32> & "C:\Program Files\mosquitto\mosquitto.exe" -c "%env:USERPROFILE\Desktop\mosquitto.conf" -v
1779612525: mosquitto version 2.1.2 starting
1779612525: Config loaded from C:\Users\hangj\Desktop\mosquitto.conf.
1779612525: Bridge support available.
1779612525: Persistence support available.
1779612525: TLS support available.
1779612525: TLS-PSK support available.
1779612525: Websockets support available.
1779612525: Opening ipv4 listen socket on port 1883.
1779612525: mosquitto version 2.1.2 running
1779612528: New connection from 172.20.10.11:64721 on port 1883.
1779612528: New client connected from 172.20.10.11:64721 as cat_bowl_01 (p4, c1, k15).
1779612528: No will message specified.
1779612528: Sending CONNACK to cat_bowl_01 (0, 0)
1779612528: Received SUBSCRIBE from cat_bowl_01
1779612528:     pet/device/control (QoS 0)
1779612528: cat_bowl_01 0 pet/device/control
1779612528: Sending SUBACK to cat_bowl_01
1779612528: Received PUBLISH from cat_bowl_01 (d0, q0, r0, m0, 'pet/device/status', ... (100 bytes))
1779612528: Received PUBLISH from cat_bowl_01 (d0, q0, r0, m0, 'pet/weight/data', ... (99 bytes))
1779612533: Received PUBLISH from cat_bowl_01 (d0, q0, r0, m0, 'pet/weight/data', ... (99 bytes))
1779612538: Received PUBLISH from cat_bowl_01 (d0, q0, r0, m0, 'pet/weight/data', ... (99 bytes))
1779612543: Received PUBLISH from cat_bowl_01 (d0, q0, r0, m0, 'pet/weight/data', ... (99 bytes))
1779612543: Received PINGREQ from cat_bowl_01
1779612543: Sending PINGRESP to cat_bowl_01
1779612548: Received PUBLISH from cat_bowl_01 (d0, q0, r0, m0, 'pet/weight/data', ... (98 bytes))
```

New client connected from 172.20.10.11 as cat_bowl_01
Received SUBSCRIBE from cat_bowl_01
pet/device/control (QoS 0)
Received PUBLISH from cat_bowl_01 ... 'pet/device/status'
Received PUBLISH from cat_bowl_01 ... 'pet/weight/data'

7.2 订阅重量数据

另开 PowerShell 窗口，使用 mosquitto_sub 订阅重量主题，并增加 -v 参数显示主题名。

```
& "C:\Program Files\mosquitto\mosquitto_sub.exe" -h localhost -t pet/weight/data -v
```

```
管理员: Windows PowerShell
Windows PowerShell
版权所有 (C) Microsoft Corporation。保留所有权利。

尝试新的跨平台 PowerShell https://aka.ms/pscore6

PS C:\Windows\system32> & "C:\Program Files\mosquitto\mosquitto_sub.exe" -h localhost -t pet/weight/data -v
pet/weight/data {"device_id":"cat_bowl_01","weight_kg":0.015000001,"raw_adc":1230,"alarm_threshold_kg":0.150000006}
pet/weight/data {"device_id":"cat_bowl_01","weight_kg":0.0085,"raw_adc":1217,"alarm_threshold_kg":0.150000006}
pet/weight/data {"device_id":"cat_bowl_01","weight_kg":0.0015,"raw_adc":1203,"alarm_threshold_kg":0.150000006}
pet/weight/data {"device_id":"cat_bowl_01","weight_kg":0,"raw_adc":1193,"alarm_threshold_kg":0.150000006}
pet/weight/data {"device_id":"cat_bowl_01","weight_kg":0,"raw_adc":1180,"alarm_threshold_kg":0.150000006}
```

图 8 mosquitto_sub 订阅 pet/weight/data 获得重量 JSON 数据

从订阅窗口可见，重量数据以 JSON 格式传输，包含 device_id、weight_kg、raw_adc 和 alarm_threshold_kg 字段。

字段	含义	示例值
device_id	终端设备编号	cat_bowl_01
weight_kg	猫粮剩余重量	0.155

字段	含义	示例值
raw_adc	ADC 原始采样值	1509
alarm_threshold_kg	低粮告警阈值	0.150

8 Python Flask + SQLite 服务端平台

8.1 PC 平台目录

PC 端平台目录为 C:\CatFoodM2M。服务端由 Flask、Paho-MQTT 和 SQLite 组成，页面模板放在 templates 目录。

```
C:\CatFoodM2M
├─ app.py          # Flask + MQTT + SQLite 服务端
├─ requirements.txt # Python 依赖列表
├─ templates
│   └─ index.html  # 前端页面
```

requirements.txt

```
flask
paho-mqtt
```

8.2 服务端功能

- 订阅 pet/weight/data，解析终端上传的重量与 raw_adc。
- 订阅 pet/device/status，更新设备在线状态、阈值和采样间隔。
- 将重量数据写入 weight_record 表，将状态写入 device_status 表。
- 当 weight_kg 小于 alarm_threshold_kg 时，写入 alarm_record 表。
- 提供 /api/latest、/api/history、/api/alarms 和 /api/control 接口。
- 通过 /api/control 将 Web 前端配置转为 MQTT 控制消息，下发到 pet/device/control。

接口	方法	用途
/	GET	返回 Web 前端页面
/api/latest	GET	返回最新重量、状态、阈值和告警状态
/api/history?limit=30	GET	返回最近重量记录，用于趋势图
/api/alarms	GET	返回最近低粮告警记录
/api/control	POST	下发采集间隔、报警阈值和清零控制指令

8.3 运行服务端

```
cd C:\CatFoodM2M
python app.py
```

服务启动后，控制台输出 [MQTT] Connected, rc = 0，说明 Flask 服务已经成功连接本机 Mosquitto，并开始订阅数据主题。浏览器访问 <http://127.0.0.1:5000> 即可打开监测页面。

9 Web 前端页面设计与运行效果

9.1 页面模块

Web 前端采用单页布局，风格简洁，包含顶部渐变标题栏、四个状态卡片、实时重量柱状趋势图、远程配置表单和告警记录表格。页面通过 JavaScript 定时请求 Flask API，每 2 秒刷新一次。

- 当前重量：展示 weight_kg，单位 kg。
- ADC 原始值：展示 raw_adc，便于观察传感器输出变化。
- 报警阈值：展示 alarm_threshold_kg。
- 设备状态：在线、离线或低粮告警。
- 实时趋势：以柱状图显示最近 30 条重量记录。
- 远程配置：支持设置采集间隔、报警阈值和清零。
- 告警记录：显示 LOW_FOOD 类型记录。

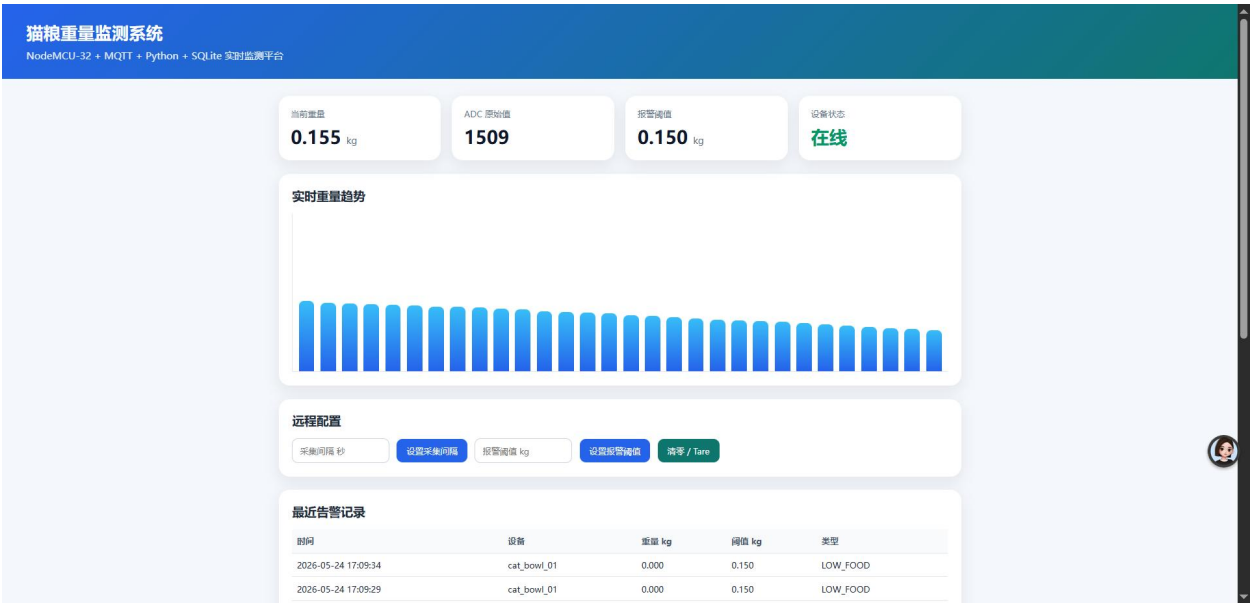


图9 Flask Web 前端实时监测页面

9.2 前端刷新与控制逻辑

页面定时刷新

```
async function refreshAll() {
  await fetchLatest();
  await fetchHistory();
  await fetchAlarms();
}

refreshAll();
setInterval(refreshAll, 2000);
```

前端下发报警阈值

```
await fetch("/api/control", {
  method: "POST",
  headers: {"Content-Type": "application/json"},
  body: JSON.stringify({
    device_id: "cat_bowl_01",
    cmd: "set_config",
    alarm_threshold_kg: Number(value)
  })
});
```


10 远程控制与功能测试

10.1 控制指令格式

Web 页面提交配置后，Flask 服务端将其封装为 JSON 并发布到 pet/device/control。NodeMCU-32 订阅该主题，解析命令后更新采集间隔、报警阈值或执行清零。

控制功能	MQTT 控制消息	终端动作
设置采集间隔	{"device_id":"cat_bowl_01","cmd":"set_config","sample_interval_s":10}	更新 sampleInterval，后续按 10 秒上传
设置报警阈值	{"device_id":"cat_bowl_01","cmd":"set_config","alarm_threshold_kg":0.2}	更新 alarmThresholdKg 并返回配置确认
清零 / 去皮	{"device_id":"cat_bowl_01","cmd":"tare"}	记录当前 ADC 作为 zeroAdc

10.2 功能测试结果

测试项	操作方法	预期结果	实验结果
Wi-Fi 连接	NodeMCU-32 启动后连接热点 hajimi	串口显示 WiFi connected 和终端 IP	通过
MQTT 连接	Mosquitto 以配置文件启动，终端连接 172.20.10.4:1883	Broker 日志显示 cat_bowl_01 connected	通过
重量数据上传	订阅 pet/weight/data	收到 weight_kg 和 raw_adc JSON 数据	通过
状态上报	订阅 pet/device/status	收到 online/config_updated/tare_done 等状态	通过
采集间隔配置	Web 页面设置采集间隔	终端接收控制并改变发布频率	通过
阈值配置	Web 页面设置报警阈值	后续数据中的阈值更新	通过
低粮告警	重量低于阈值	页面显示低粮告警并写入告警表	通过
Web 前端	浏览器打开 127.0.0.1:5000	页面 200 返回并实时刷新	通过

11 问题排查与解决过程

11.1 PlatformIO 首次安装较慢

PlatformIO 第一次创建 ESP32 工程时需要下载 espressif32 平台包、编译工具链、Arduino-ESP32 框架和工具包，因此 Project Wizard 会显示 Please wait。处理方法是等待下载完成，必要时切换网络或使用代理，并将项目放在无中文、无空格的路径中。

11.2 mqtt_server 变量未定义

首次编译时出现 mqtt_server was not declared in this scope。原因是代码中 client.setServer(mqtt_server, 1883) 使用了该变量，但前面没有定义或名称不一致。解决方法是在 Wi-Fi 参数后增加 const char* mqtt_server = "172.20.10.4"，并保证变量名完全一致。

11.3 NodeMCU-32 MQTT rc=-2

串口显示 Wi-Fi 已连接但 MQTT failed, rc=-2，说明终端无法连接 Broker。排查发现 Mosquitto 默认以 local only mode 运行，只监听 127.0.0.1:1883，而 NodeMCU-32 是从局域网 IP 172.20.10.11 连接 PC。因此需要停止后台服务，并使用 listener 1883 0.0.0.0 重新启动。

11.4 1883 端口被占用

使用 netstat -ano | findstr :1883 发现 1883 被 mosquitto.exe 服务进程占用，只监听 127.0.0.1。通过 Stop-Service mosquitto 和 Set-Service mosquitto -StartupType Manual 停止服务，再以自定义配置启动 Broker，问题解决。

11.5 iPhone 热点连接注意事项

NodeMCU-32 与 PC 必须连接同一热点。实验中热点名称为 hajimi，终端获得 IP 172.20.10.11，PC 端 Broker 地址为 172.20.10.4。若使用中文热点名，ESP32 可能连接不稳定，建议改为英文名，并在 iPhone 中开启“最大兼容性”。

12 实验结果与总结

本实验完成了基于 NodeMCU-32 的猫粮重量监测 M2M 系统全流程实现。终端侧完成 FSR 分压采样、ADC 数据处理、Wi-Fi 接入、MQTT 连接和 JSON 数据发布；服务端完成 Mosquitto Broker 局域网监听、Flask MQTT 订阅、SQLite 持久化存储、低粮告警判断和 Web API 提供；应用层实现了简洁清爽的 Web 前端，用于展示实时重量、ADC 原始值、报警阈值、设备状态、趋势图和告警记录，并支持远程控制。

通过本实验可以验证，MQTT 发布/订阅模型适合资源受限的 M2M 终端与 PC 服务平台之间的轻量级通信。NodeMCU-32 作为终端设备，能够在局域网内稳定接入 Broker 并持续上传数据；PC 端通过 Python 和 SQLite 可以快速构建可视化与数据管理平台。系统已经形成“数据上行 + 控制下行”的闭环，满足课程实验对设备管理、资源管理和应用展示的要求。

后续优化方向包括：增加多点标定页面、加入告警冷却时间避免重复告警、使用 WebSocket 进一步降低刷新延迟、支持多设备 device_id 管理、加入 MQTT 账号密码和 TLS 加密，以及将 FSR 升级为称重传感器 + HX711 以提高称重精度。

附录 A 关键配置与代码片段

A.1 platformio.ini

```
[env:nodemcu-32s]
platform = espressif32
board = nodemcu-32s
framework = arduino
monitor_speed = 115200

lib_deps =
    knolleary/PubSubClient@^2.8
    bblanchon/ArduinoJson@^6.21.5
```

A.2 Mosquitto 配置

```
listener 1883 0.0.0.0
allow_anonymous true
```

A.3 Flask 服务端核心处理

```
def on_message(client, userdata, msg):
    payload = msg.payload.decode("utf-8")
    data = json.loads(payload)

    if msg.topic == "pet/weight/data":
        save_weight(data)
    elif msg.topic == "pet/device/status":
        save_status(data)

@app.route("/api/control", methods=["POST"])
def api_control():
    data = request.json or {}
    payload = {"device_id": "cat_bowl_01", "cmd": data.get("cmd", "set_config")}
    if "sample_interval_s" in data:
        payload["sample_interval_s"] = int(data["sample_interval_s"])
    if "alarm_threshold_kg" in data:
        payload["alarm_threshold_kg"] = float(data["alarm_threshold_kg"])
    mqtt_client.publish("pet/device/control", json.dumps(payload), qos=0)
    return jsonify({"success": True, "payload": payload})
```


A.4 前端主要 API

API	用途
/api/latest	获取最新重量、ADC、阈值、设备状态
/api/history?limit=30	获取最近 30 条重量记录用于趋势图
/api/alarms	获取最近告警记录
/api/control	向终端下发控制指令